# DATA STRUCTURES ASSIGNMENT

**Name:** Bhoomika Reddy
**Regno:** 192472379

## Q1. Define non-primitive data structures and classify them (5 marks)

**Definition:** Non-primitive data structures are derived data structures built from primitive data types (int, char, float, double, etc.). They store a collection of values, possibly of different types, and define how the data is organised and accessed. Examples: arrays, linked lists, stacks, queues, trees and graphs. Unlike primitive types, they are not supplied directly by the machine and are created by the programmer or the language library.

### Classification

**1. Linear data structures** - elements are arranged sequentially; each element has one predecessor and one successor (except the first and last).

- **Array:** fixed-size collection of same-type elements in contiguous memory; O(1) indexed access.
- **Linked list:** nodes connected by pointers; dynamic size. Types: singly, doubly, circular.
- **Stack:** LIFO; operations push and pop at one end (top).
- **Queue:** FIFO; insertion at rear, deletion at front. Types: simple, circular, priority queue, deque.

**2. Non-linear data structures** - elements are not in a sequence; one element may connect to many others.

- **Tree:** hierarchical structure with a root and child nodes (binary tree, BST, AVL tree, heap).
- **Graph:** set of vertices connected by edges (directed/undirected, weighted/unweighted).

**Other ways of classifying**

| Basis | Type | Example |
|---|---|---|
| Memory allocation | Static (fixed size) | Array |
| | Dynamic (grows/shrinks) | Linked list, tree |
| Data type of elements | Homogeneous | Array |
| | Heterogeneous | Structure, union, class |
| Storage | Contiguous / Linked | Array / Linked list |

**Summary:** Non-primitive structures are first divided into linear and non-linear; linear ones are stored either contiguously (array) or by links (linked list), while stack and queue restrict how elements are inserted and removed.

## Q2. Implement a sparse matrix using an efficient representation and analyse space (10 marks)

**Sparse matrix:** a matrix in which most elements are zero. Storing it as a 2-D array wastes memory, so only the non-zero elements are stored along with their positions. Here the triplet (3-tuple) representation is used: each non-zero element is stored as (row, column, value), and row 0 is a header holding (total rows, total columns, number of non-zeros).

**Example (4 x 5 matrix, 5 non-zero elements)**

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 3 | 0 | 0 |
| 0 | 0 | 0 | 0 | 4 |
| 0 | 7 | 0 | 0 | 0 |
| 5 | 0 | 0 | 2 | 0 |

Triplet form:

| Row | Column | Value |
|---|---|---|
| 4 | 5 | 5 |
| 0 | 2 | 3 |
| 1 | 4 | 4 |
| 2 | 1 | 7 |
| 3 | 0 | 5 |
| 3 | 3 | 2 |

### Procedure to build the representation

- Scan the matrix row by row, left to right.
- Whenever an element is non-zero, store its row number, column number and value as the next triplet.
- Skip all zero elements; they take no space.
- Finally fill row 0 (header) with the total rows, total columns and the count of non-zero elements.
- To read an element (i, j), search the triplets for that row and column; if not found, the value is 0.

### Space analysis

Let the matrix be m x n with t non-zero elements, and each value take one unit of memory.

- **Normal 2-D array:** m x n units. For the example: 4 x 5 = 20 units.
- **Triplet representation:** 3 x (t + 1) units (t terms plus the header). For the example: 3 x 6 = 18 units.
- **Condition for saving space:** $3(t + 1) < m \times n$, i.e. $t < (m \times n)/3 - 1$. The gain grows as the matrix gets sparser; e.g. a 100 x 100 matrix with 100 non-zeros needs 303 units instead of 10,000 (about 97% saving).
- **Time:** building the triplet takes O(m x n); traversing it takes O(t). Compressed Sparse Row (CSR) needs $2t + m + 1$ units and gives faster row access.

# DATA STRUCTURES ASSIGNMENT

## Q3. Describe ADTs in operating systems (5 marks)

**Definition:** An Abstract Data Type (ADT) is a mathematical model of a data structure that is defined by the data it holds and the operations allowed on it, without revealing how it is implemented. Users see only the interface (what it does), while the implementation (how it does it) stays hidden. This gives encapsulation, modularity and the freedom to change the implementation without changing the code that uses it.

### ADTs used inside an operating system

| ADT | Operations | Use in the operating system |
|---|---|---|
| Queue | enqueue, dequeue, isEmpty | Ready queue and device queues for process scheduling (FCFS, Round Robin); print spooling; keyboard buffer |
| Priority queue | insert, extractMax/Min | Priority scheduling, real-time task scheduling, interrupt handling |
| Stack | push, pop, peek | Function call stack, saving return addresses and local variables, context handling, undo of nested interrupts |
| List / linked list | insert, delete, search, traverse | Free-space list in memory management, process control block (PCB) lists, file allocation chains |
| Tree | insert, delete, search | Directory structure of the file system, B-trees for file indexing, process tree (parent-child) |
| Hash table (map) | put, get, remove | Page tables, symbol tables, file name look-up, caches |
| Graph | add/remove vertex or edge | Resource allocation graph and wait-for graph for deadlock detection |
| Bitmap / array | set, clear, test | Free-block map of disk, frame allocation in memory |

### Benefits in an operating system

- **Abstraction:** kernel modules use a queue or table through its operations only, so the scheduler does not depend on how the queue is stored.
- **Modularity and maintainability:** an implementation (for example array to linked list) can be replaced without changing the rest of the kernel.
- **Reliability and protection:** hiding internal data prevents accidental corruption of kernel structures.
- **Reusability:** the same ADT (such as a queue) serves the CPU scheduler, disk scheduler and I/O buffering.

**Example:** In Round Robin scheduling, the ready queue is a Queue ADT. The scheduler calls dequeue() to pick the next process, runs it for one time quantum, then calls enqueue() to put it back at the rear if it has not finished. The scheduler never needs to know whether the queue is an array or a linked list.